

מסמך טכני - QuizProj

1. פרטי הפרויקט וקישורים

שם הפרויקט: QuizProj - מערכת מבחנים צד לקוח

טכנולוגיות: Node.js, `localStorage`, JSON, OOP, ES Modules, JavaScript, CSS, HTML ו-Express.

• קוד מקור ב-GitHub: <https://github.com/eбраheem222-tech/QuizProj>

• אתר GitHub Pages: <https://eбраheem222-tech.github.io/QuizProj>

• כתובת בהרצה מקומית: `http://localhost:3000`

המערכת שומרת את המידע בדפדפן בלבד. שרת Express אינו שומר מידע ואינו כולל בסיס נתונים; תפקידו להגיש את תיקיית `client/` כאתר סטטי ולספק כתובות URL נקיות.

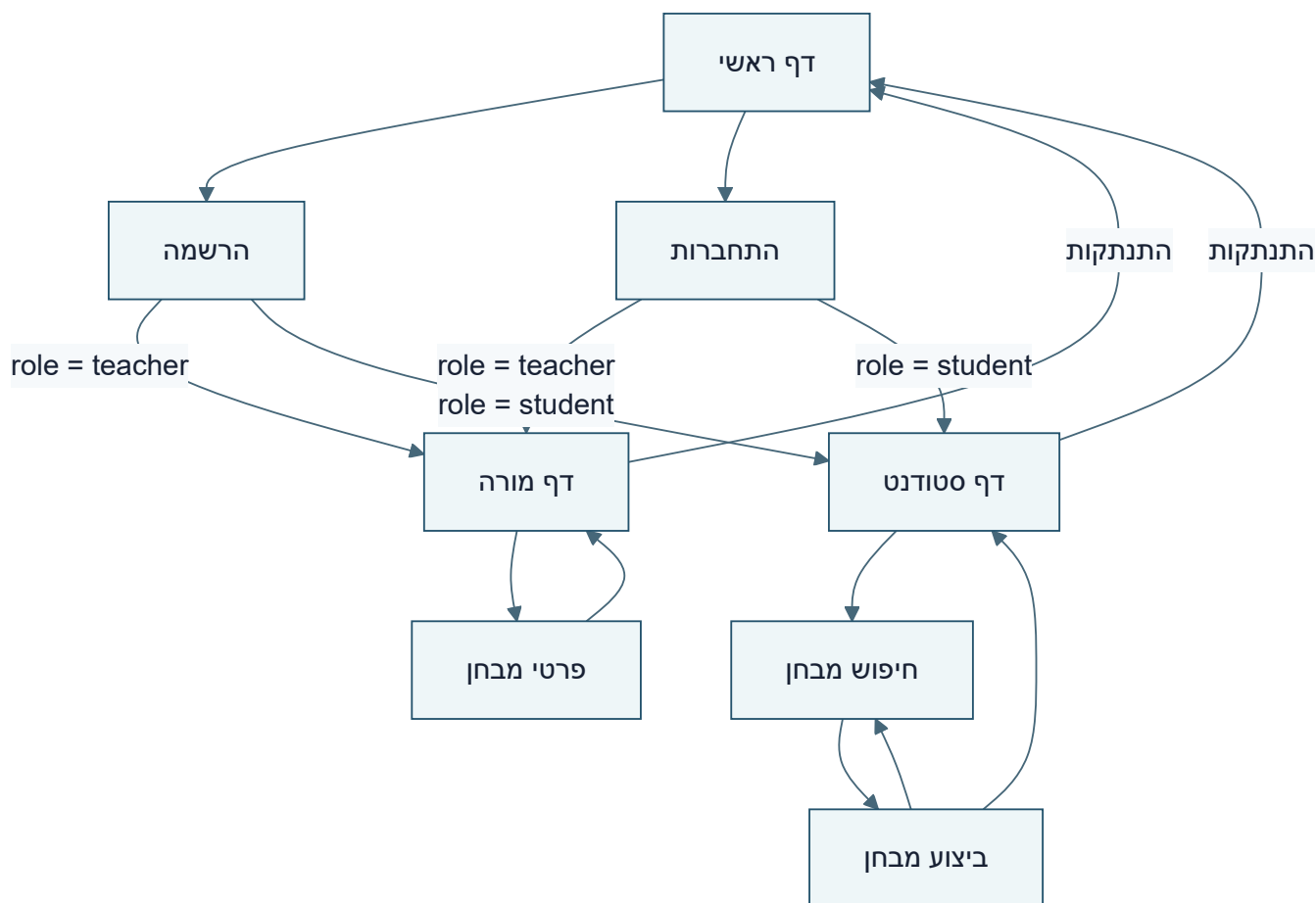
הרצה מקומית

```
npm install
npm start
```

לאחר מכן פותחים בדפדפן את `http://localhost:3000`. אין לפתוח את הקבצים ישירות באמצעות `file://`, מפני ש-ES Modules ונתיבים כגון `/register` דורשים שרת HTTP.

2. דפי האתר והניווט ביניהם

דף	נתיב Express מקומי	נתיב סטטי ב-GitHub Pages	הרשאה	תפקיד הדף
דף ראשי	/	/QuizProj/client/index.html	כולם	הצגת הפרויקט וקישורים להרשמה ולהתחברות
הרשמה	/register	/QuizProj/client/pages/register.html	אורח	יצירת משתמש מסוג מורה או סטודנט
התחברות	/login	/QuizProj/client/pages/login.html	אורח	התחברות באמצעות אימייל או תעודת זהות
דף מורה	/teacher	/QuizProj/client/pages/teacher.html	מורה	יצירה, חיפוש, מחיקה, ייבוא וייצוא של מבחנים
פרטי מבחן	/exam/:id	/QuizProj/client/pages/exam-details.html?id=:id	מורה בעל המבחן	עריכת מבחן, ניהול שאלות וצפייה בתוצאות
דף סטודנט	/student	/QuizProj/client/pages/student.html	סטודנט	היסטוריית מבחנים, ציונים וממוצע
חיפוש מבחן	/search	/QuizProj/client/pages/search.html	סטודנט	חיפוש לפי שם, תיאור, קטגוריה או קוד
ביצוע מבחן	/take/:id	/QuizProj/client/pages/take-exam.html?id=:id	סטודנט	מענה על שאלות, טיימר ושליחת התוצאה



המודול `router.js` יוצר את הכתובת המתאימה לסביבת ההרצה. בהרצה עם Node הוא מחזיר נתיב נקי, לדוגמה `/exam/123`. ב-GitHub Pages הוא מחזיר נתיב לקובץ HTML עם מזהה ב-query string, לדוגמה `exam-details.html?id=123`.

3. פורמט הנתונים הנשמרים כ-JSON

המחלקה `StorageService` מוסיפה את התחילית `quizproj.` לכל מפתח, ממירה אובייקטים ל-JSON באמצעות `JSON.stringify`, וקוראת אותם באמצעות `JSON.parse`.

מפתח ב- <code>localStorage</code>	סוג הערך	תוכן
<code>quizproj.users</code>	מערך	משתמשים רשומים
<code>quizproj.currentUserId</code>	מחרוזת	מזהה המשתמש המחובר
<code>quizproj.exams</code>	מערך	מבחנים והשאלות שבתוכם
<code>quizproj.results</code>	מערך	ניסיונות וציוני תלמידים
<code>quizproj.theme</code>	מחרוזת	ערך <code>light</code> או <code>dark</code>

משתמש (User)

```
{
  "id": "demo-student",
  "fullName": "סטודנט הדגמה",
  "nationalId": "222222222",
  "email": "student@demo.com",
  "password": "1234",
  "role": "student",
  "createdAt": "2026-07-10T00:00:00.000Z"
}
```

- `role` יכול להיות `teacher` או `student`.
- `id` נוצר באמצעות `crypto.randomUUID()`.
- בפרויקט לימודי זה הסיסמה נשמרת כטקסט רגיל. במערכת אמיתית יש לבצע אימות בצד שרת ולשמור hash בלבד.

מבחן (Exam) ושאלה (Question)

```
{
  "id": "demo-js-exam",
  "teacherId": "demo-teacher",
  "title": "JavaScript Basics",
  "description": "מבחן הדגמה קצר",
  "category": "JavaScript",
  "code": "JS-DEMO",
  "durationMinutes": 15,
  "shuffleQuestions": true,
  "questions": [
    {
      "id": "question-id",
      "text": "localStorage מהי המטרה של?",
      "answers": [
        "שמירת נתונים בדפדפן",
        "שליחת אימיילים",
        "הרצת שרת",
        "CSS עיצוב"
      ],
      "correctAnswerIndex": 0,
      "difficulty": "easy"
    }
  ],
  "createdAt": "2026-07-10T00:00:00.000Z",
  "updatedAt": "2026-07-10T00:00:00.000Z"
}
```

- `teacherId` מקשר את המבחן למורה שיצר אותו.
- `code` הוא קוד חיפוש ייחודי למבחן.
- `correctAnswerIndex` הוא מיקום התשובה הנכונה במערך `answers`, החל מ-0.
- `difficulty` יכול להיות `easy`, `medium` או `hard`.

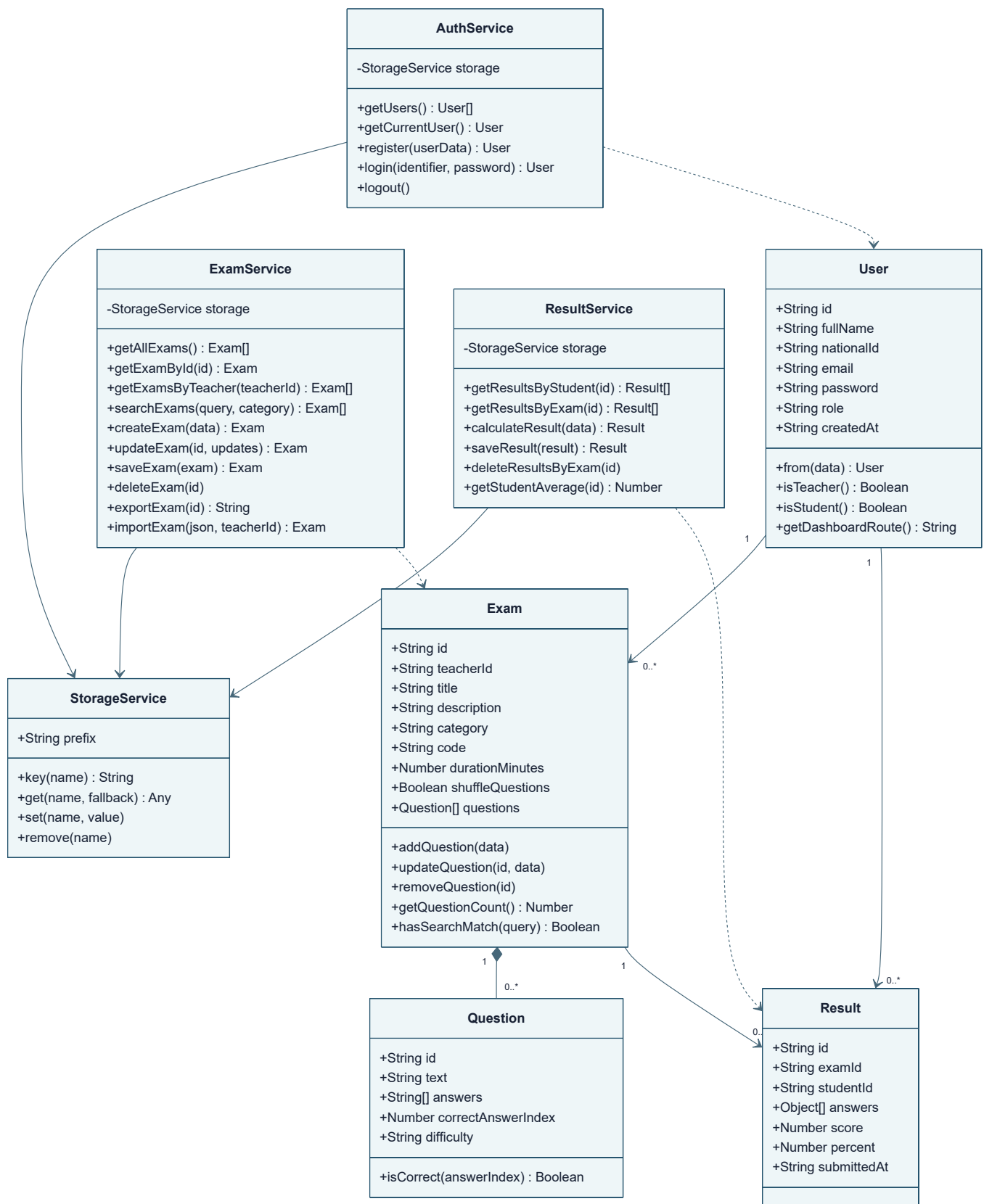
```
{
  "id": "result-id",
  "examId": "demo-js-exam",
  "examTitle": "JavaScript Basics",
  "studentId": "demo-student",
  "studentName": "סטודנט הדגמה",
  "answers": [
    {
      "questionId": "question-id",
      "questionText": "localStorage מהי המטרה של?",
      "answerIndex": 0,
      "selectedAnswer": "שמירת נתונים בדפדפן",
      "correctAnswerIndex": 0,
      "correctAnswer": "שמירת נתונים בדפדפן",
      "isCorrect": true
    }
  ],
  "score": 1,
  "totalQuestions": 1,
  "percent": 100,
  "startedAt": "2026-07-10T10:00:00.000Z",
  "submittedAt": "2026-07-10T10:04:00.000Z",
  "durationSeconds": 240
}
```

בתוצאה נשמר snapshot של נוסח השאלה, התשובה שנבחרה והתשובה הנכונה. לכן ניתן להציג ניסיון ישן גם אם המורה עורך את המבחן לאחר ההגשה.

קשרים בין הנתונים

- Exam.teacherId מפנה אל User.id של מורה.
- Result.examId מפנה אל Exam.id.
- Result.studentId מפנה אל User.id של סטודנט.
- Result.answers[].questionId מפנה אל Question.id שהיה קיים בזמן ההגשה.

4. המחלקות העיקריות - UML



אחריות המחלקות

• **User**, **Exam**, **Question**, **Result** : מודלי הנתונים והפעולות ששייכות לאובייקט עצמו.

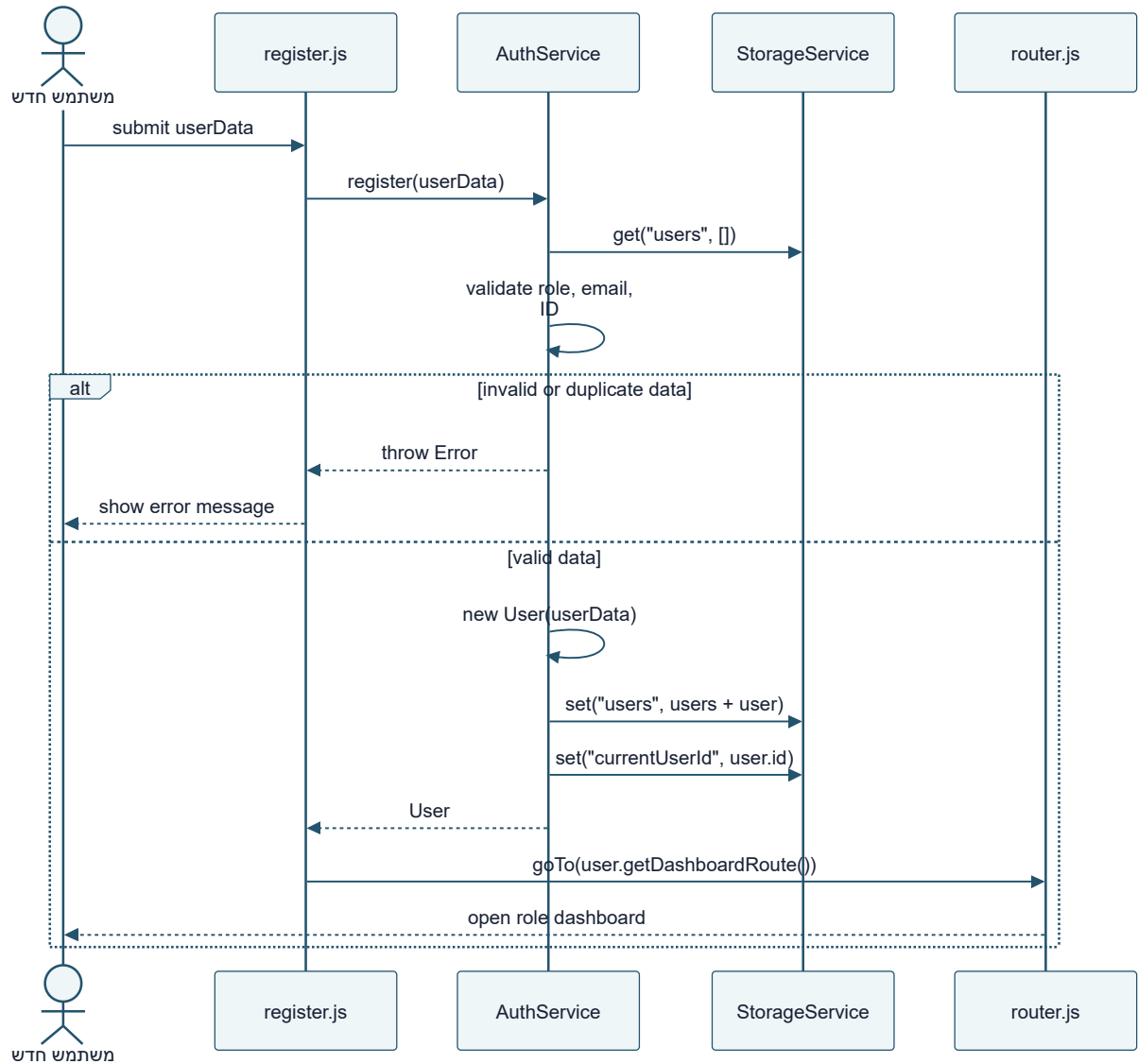
- `StorageService` : שכבת גישה יחידה ל- `localStorage` ולטיפול ב-JSON.
- `AuthService` : הרשמה, התחברות, התנתקות וקבלת המשתמש הנוכחי.
- `ExamService` : יצירה, עריכה, חיפוש, מחיקה, ייבוא וייצוא של מבחנים.
- `ResultService` : חישוב ציון, שמירת ניסיון, שליפת היסטוריה וחישוב ממוצע.
- מודולי `pages/` : קוראים את הטפסים והאירועים מה-DOM ומפעילים את השירות המתאים.

5. תזרימי מערכת מרכזיים

כל התזרימים הבאים נבדקו מול מודולי הדפים, השירותים והמודלים בפועל.

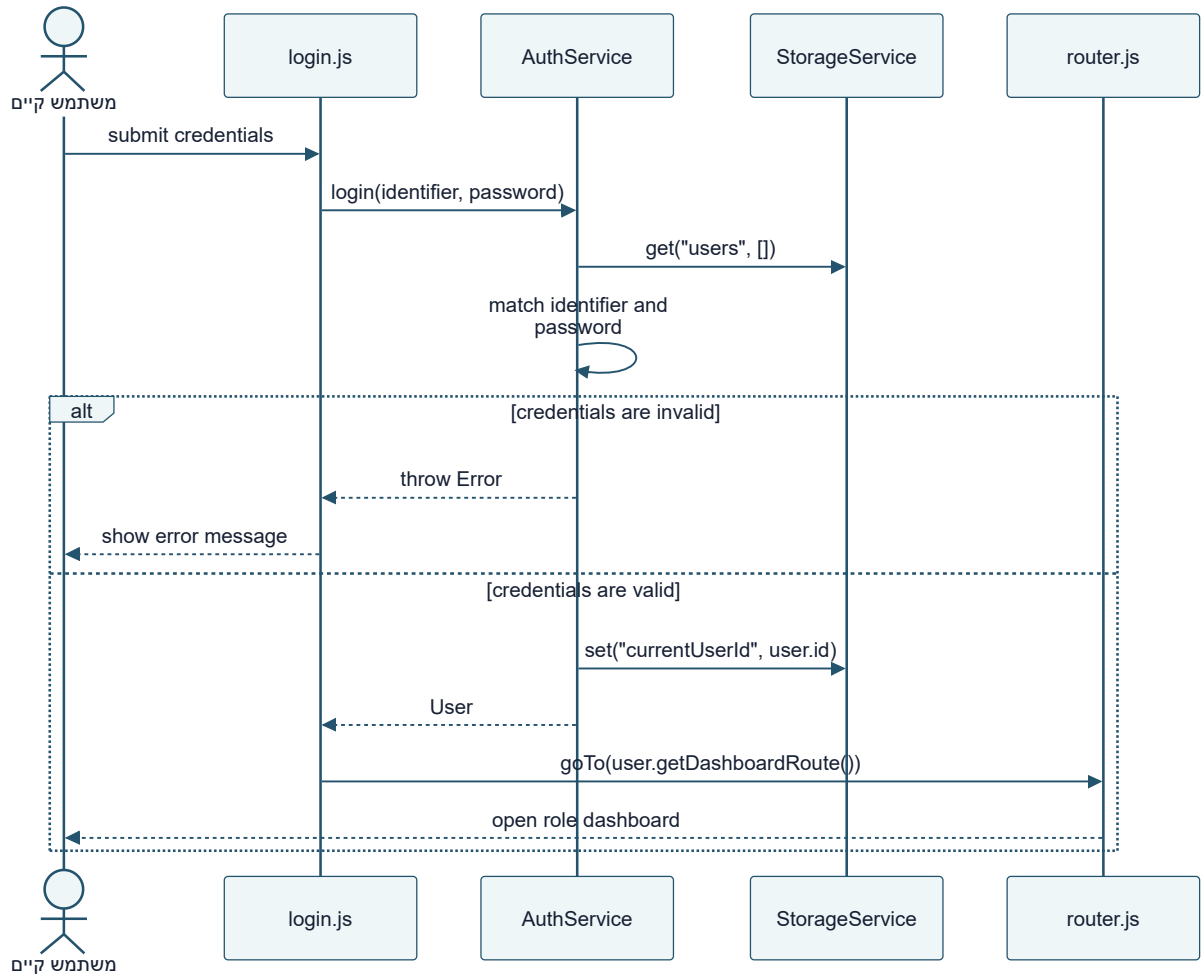
תהליך שנבדק	תוצאה	הערה
הרשמה והתחברות	תקין	נשמר <code>currentUserId</code> ומתבצעת הפניה לפי <code>role</code>
הגנת דפים לפי תפקיד	תוקן ואומת	משתמש חסר מופנה להתחברות; תפקיד שגוי מופנה לדשבורד שלו ומודול הדף מקבל <code>null</code>
יצירת מבחן	תוקן בתרשים	לאחר יצירה הרשימה מתרעננת; אין מעבר אוטומטי לדף הפרטים
עריכת מבחן ושאלות	תקין	נבדקת בעלות המורה לפני רינדור ונשמר המבחן המעודכן
מחיקת מבחן	תקין	מפעיל בנפרד מחיקת מבחן ומחיקת תוצאות <code>teacher.js</code>
חיפוש וביצוע מבחן	תקין	מוצגים רק מבחנים שיש בהם לפחות שאלה אחת
טיימר ושליחת תשובות	תקין	סיום זמן שולח אוטומטית; בשליחה ידנית יש אישור לתשובות חסרות
היסטוריה ותוצאות מורה	תקין	תוצאות מסוננות לפי סטודנט או מבחן וממוינות מהחדש לישן

FLOW 1 - הרשמה והפניה לפי תפקיד



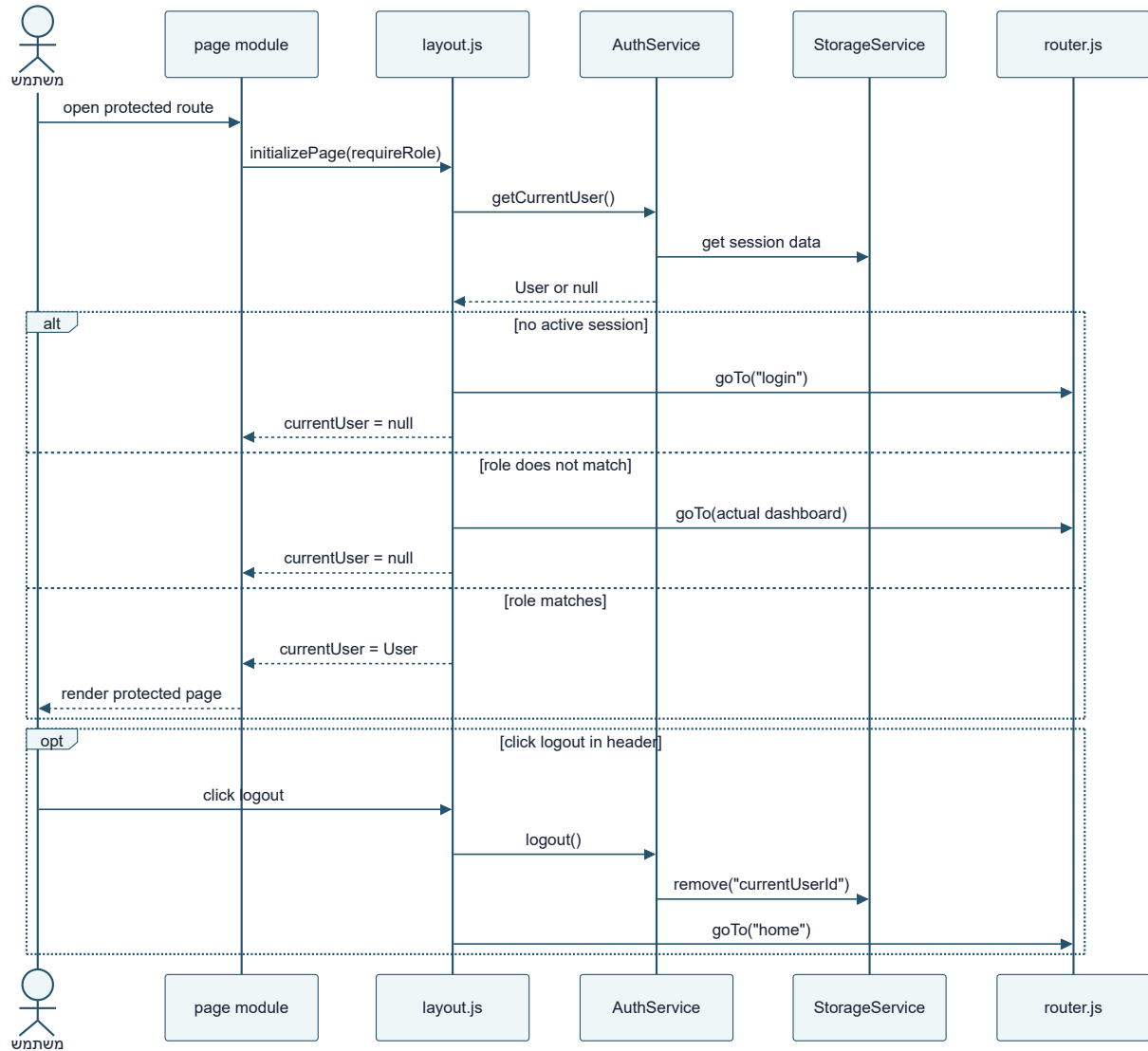
המידע שעובר: אובייקט `userData` עם שם, תעודת זהות, אימייל, סיסמה ותפקיד. בהצלחה מוחזר `User`; בכישלון נזרק `Error` ולא נשמר מידע חלקי.

FLOW 2 - התחברות והפניה לדשבורד



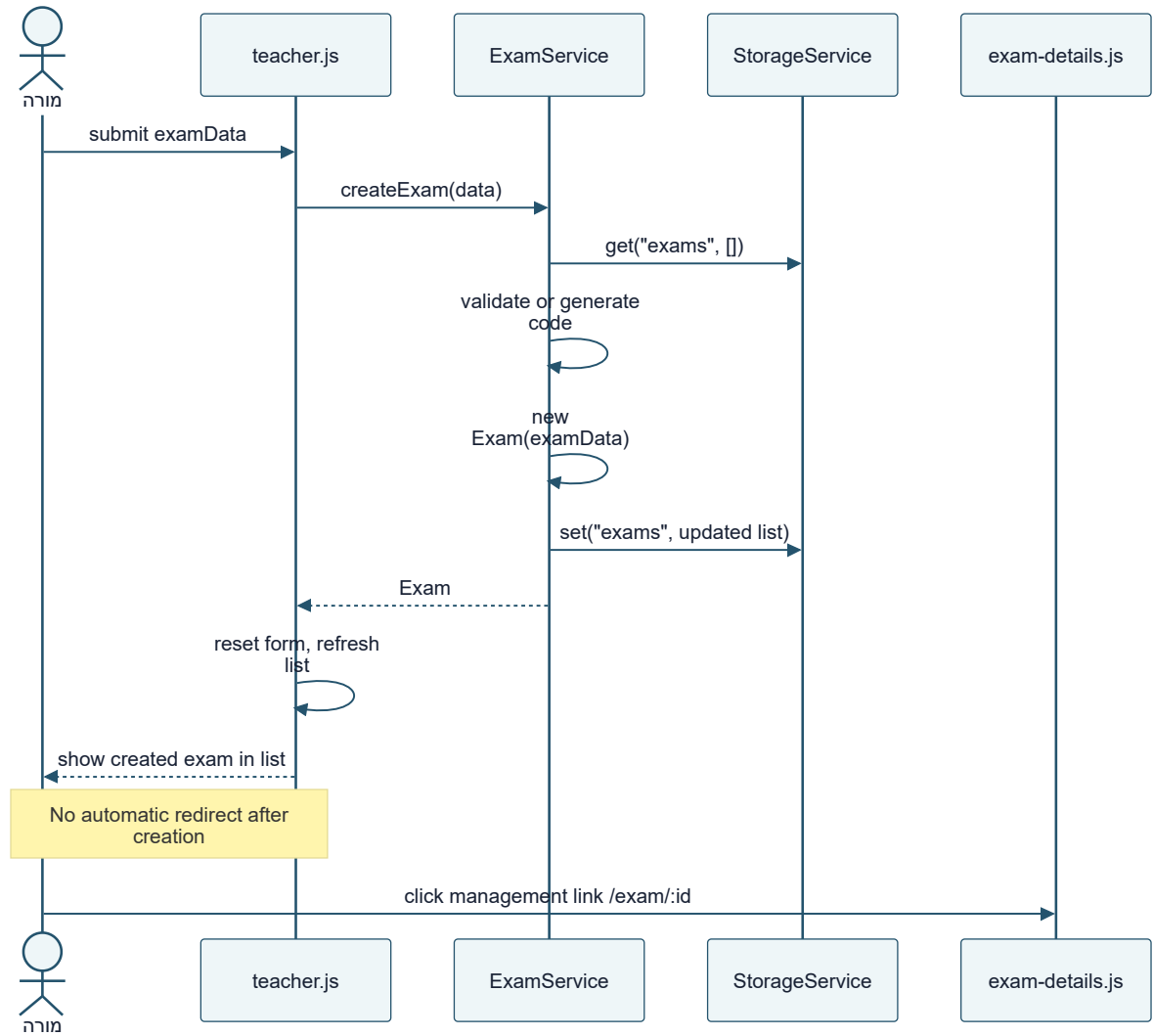
`identifier` יכול להיות אימייל או תעודת זהות. הדשבורד נקבע על ידי `User.getDashboardRoute()`.

FLOW 3 - הגנת דף לפי תפקיד והתנתקות



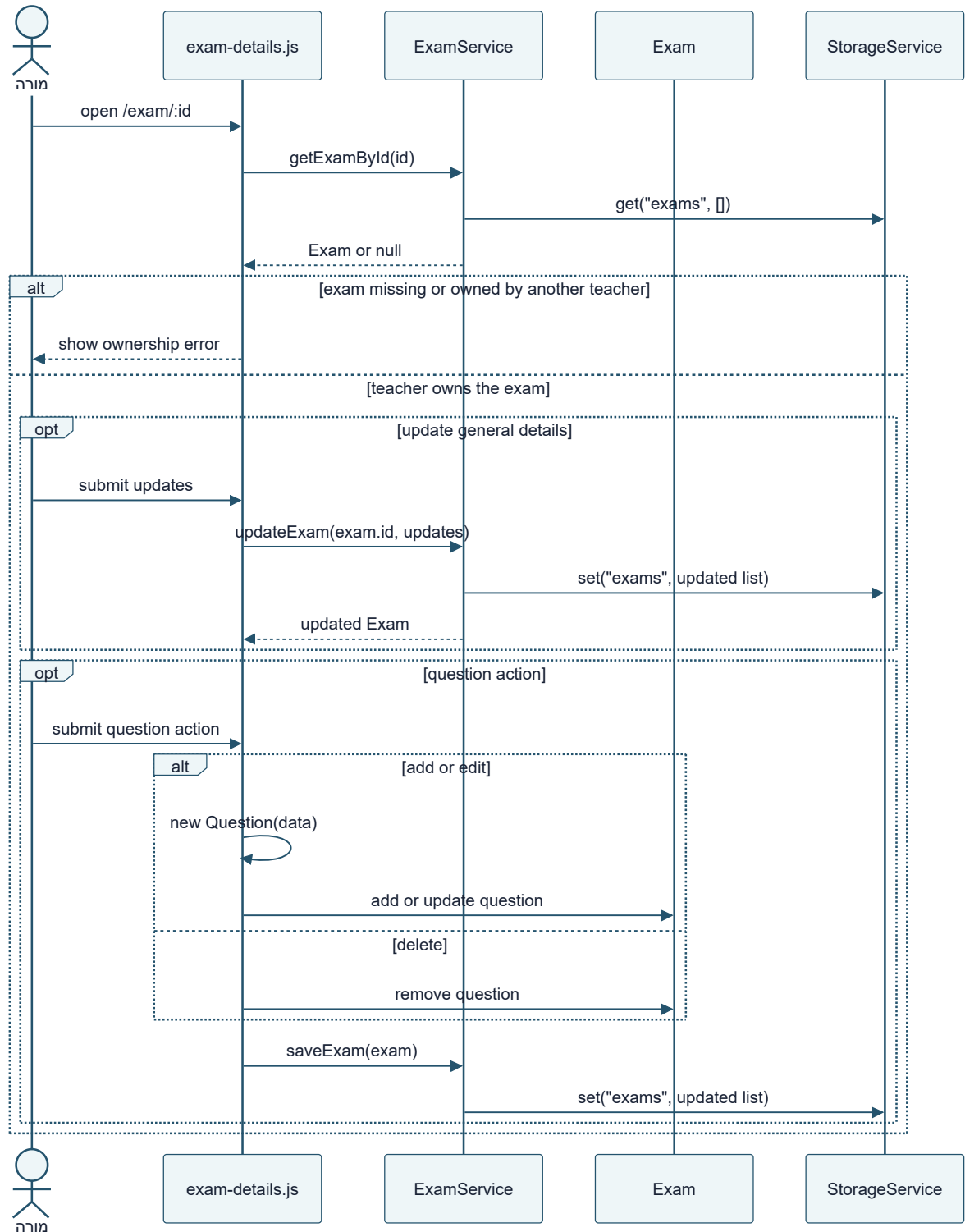
החזרת `currentUser = null` במקרה של תפקיד שגוי מונעת ממודול הדף להתחיל לרנדור בזמן שמתבצע ה-`redirect`.

FLOW 4 - מורה יוצר מבחן



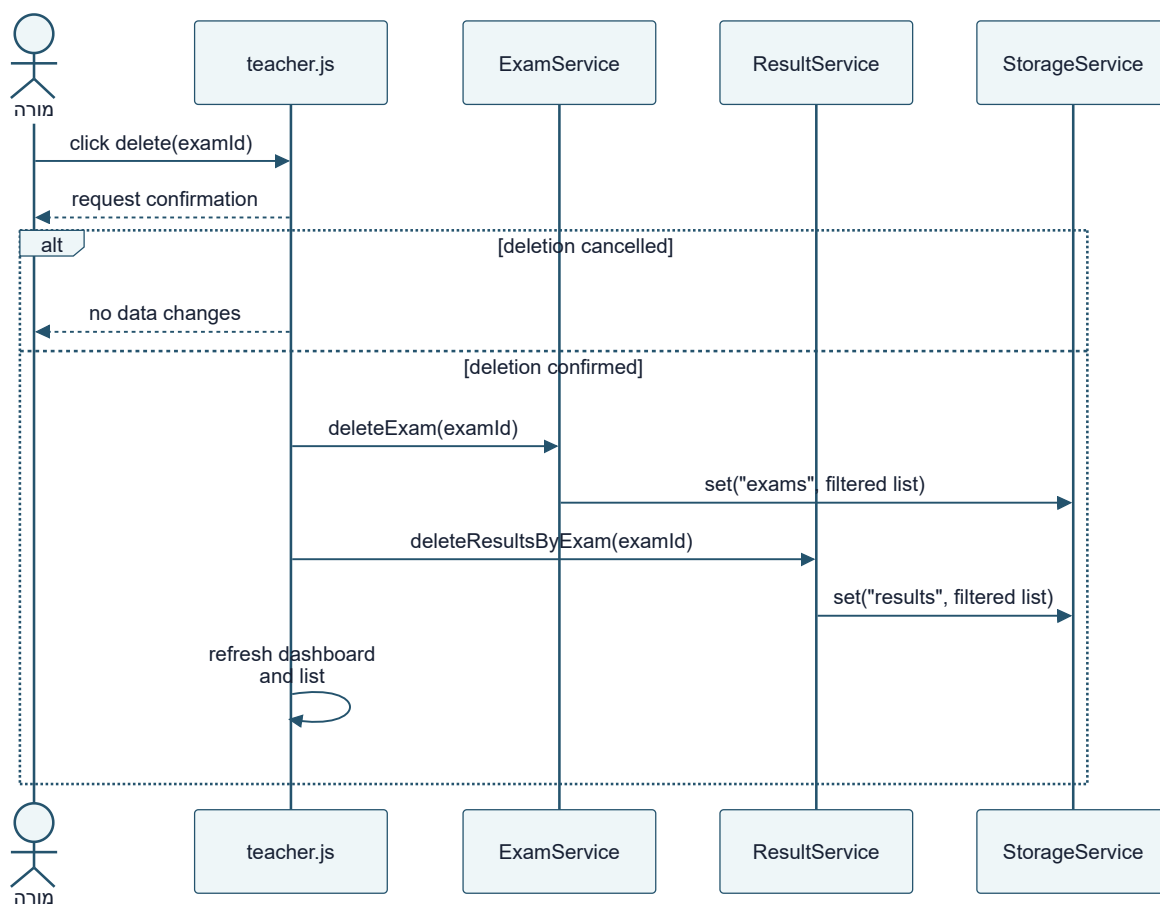
examData כולל teacherId, שם, תיאור, קטגוריה, קוד, משך והגדרת ערבוב. אם הקוד ריק השירות יוצר קוד ייחודי.

FLOW 5 - מורה עורך מבחן ומנהל שאלות



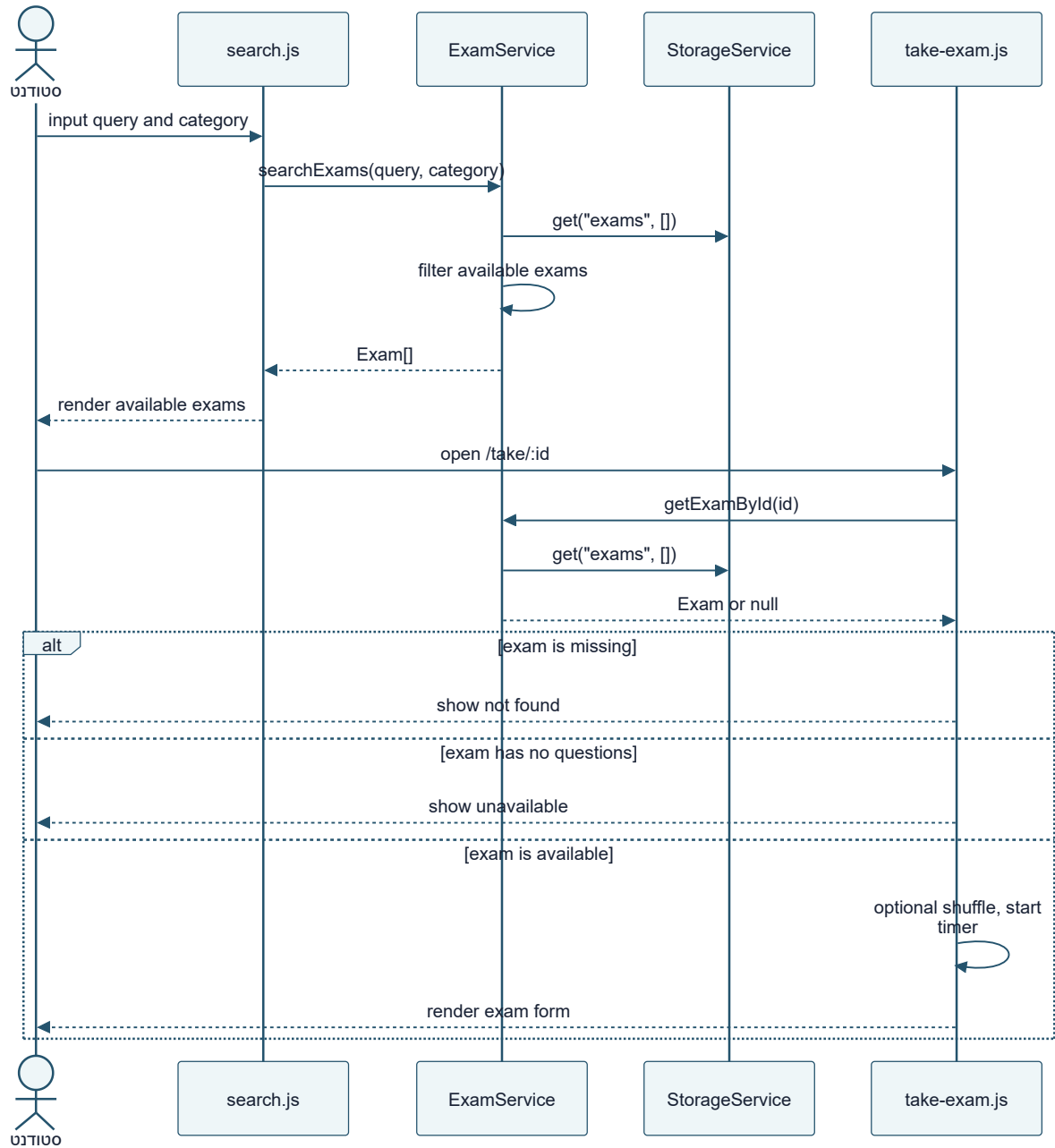
המזהה מגיע מ- `/exam/:id` ב-Express או מהפרמטר `?id=` ב-GitHub Pages. בדיקת הבעלות מתבצעת לפני הצגת טפסי הניהול.

FLOW 6 - מורה מוחק מבחן ותוצאות



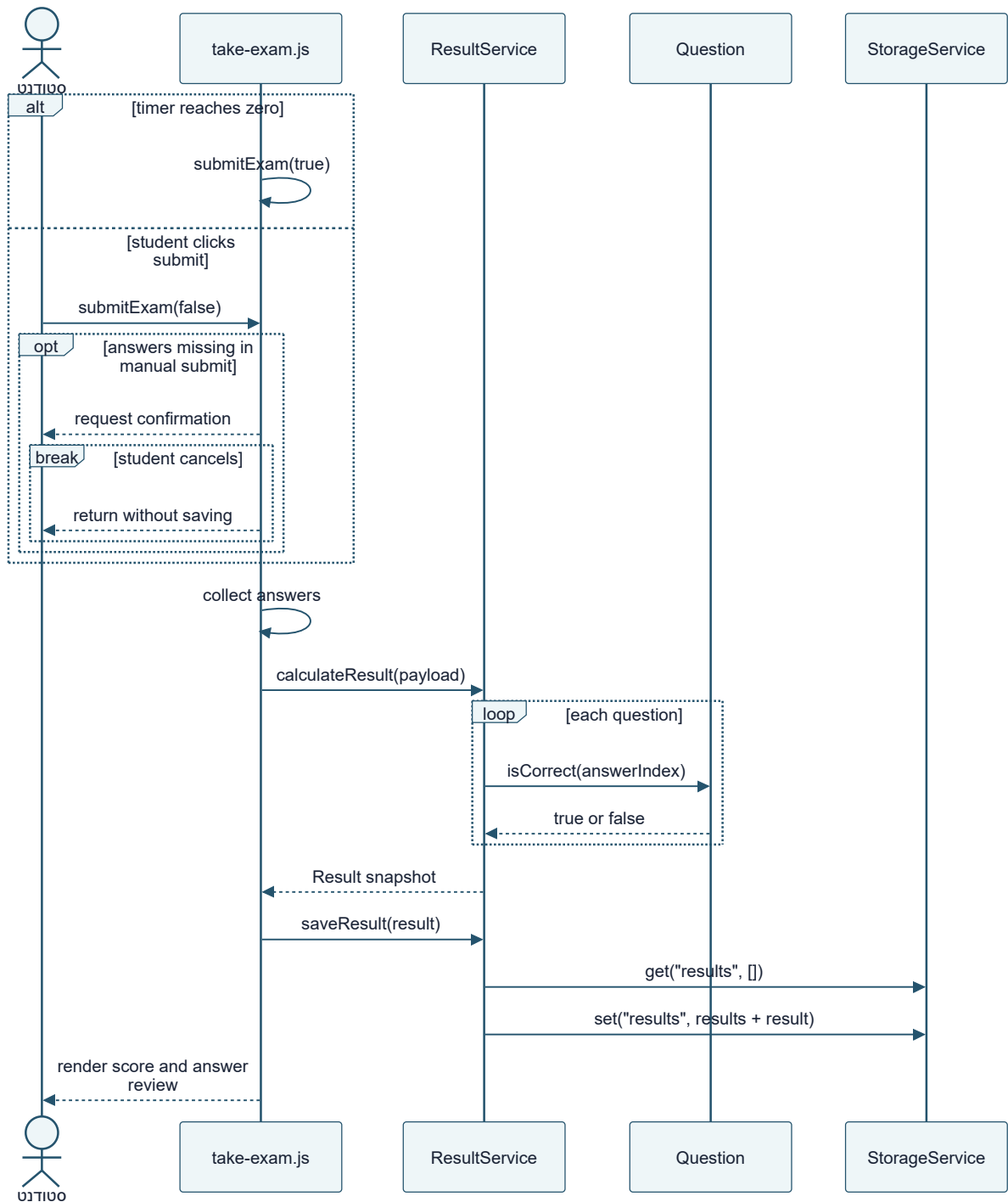
המחיקה המדורגת מתוזמרת ב- `teacher.js : ExamService.deleteExam()` אינו מוחק תוצאות בעצמו, ולכן הדף מפעיל גם את `ResultService`.

FLOW 7 - סטודנט מחפש ופותח מבחן



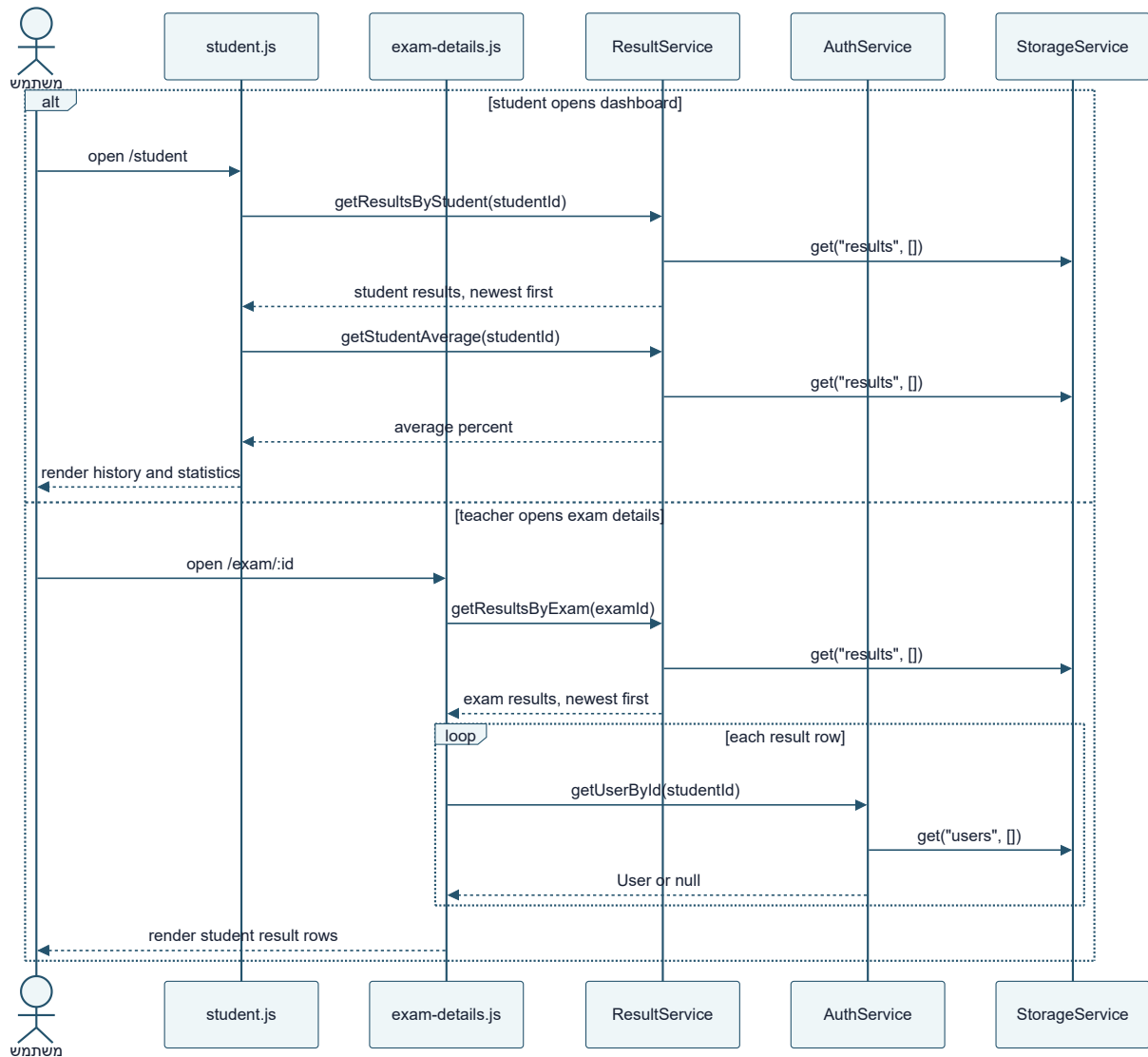
מחפש בשם, בתיאור, בקטגוריה ובקוד, ומסיר מראש מבחנים ללא שאלות. `searchExams()`

FLOW 8 - שליחת מבחן, חישוב ושמירת ציון



המפה שנשלחת לחישוב היא `{ questionId: answerIndex }`. אם הסטודנט מבטל את אישור השליחה החסרה, הפונקציה נעצרת לפני חישוב או שמירה.

FLOW 9 - היסטוריית סטודנט ותוצאות מורה



התוצאה שומרת snapshot של השאלה והתשובות, ולכן היסטוריית הסטודנט נשארת קריאה גם אם המבחן נערך לאחר ההגשה.

6. מבנה הפרויקט

```
QuizProj/
|-- server.js           # app.get ונתיבי Express שרת
|-- package.json       # npm תלויות ופקודות
|-- README.md          # הוראות ופיצ'רים
|-- TECHNICAL_DOCUMENT.md # מסמך זה
|-- client/
|   |-- index.html
|   |-- pages/         # דפי מורה, סטודנט, הרשמה, התחברות ומבחנים
|   |-- assets/
|       |-- css/styles.css
|       |-- js/
|           |-- models/ # User, Exam, Question, Result מחלקות
|           |-- services/ # Auth, Exam, Result, Storage, Seed
|           |-- pages/   # לכל דף DOM לוגיקה
|           |-- ui/      # תפריט, מצב כהה והודעות
|           |-- utils/   # וייצוא קבצים HTML, ניווט
|-- index.html         # GitHub Pages עבור client הפניה לתיקית
```

7. הערות ארכיטקטורה

- המערכת פועלת ללא Backend עסקי: אין API ואין מסד נתונים.
- Express מוגדר ב- `server.js` עם `express.static(clientDir)` ועם `app.get` לכל דף.
- ה-DI מתבצע באמצעות בנאי השירותים, לדוגמה `new AuthService(storage)`. כאשר לא מעבירים שירות אחסון, נוצר `StorageService` כברירת מחדל.
- `SeedService` מוסיף משתמשי הדגמה ומבחן לדוגמה רק כאשר האחסון ריק.
- גישה לדפי תפקיד נבדקת מול המשתמש המחובר. מורה יכול לנהל רק מבחן שה- `teacherId` שלו שווה למזהה המורה.
- מחיקת מבחן כוללת גם מחיקת התוצאות המקושרות אליו.